Synchronization exercises

State assumptions and intermediate reasoning before consulting the separate solutions.

OS05-E01 Critical-section properties

A lock guarantees mutual exclusion but lets completed participants overtake one contender indefinitely.

1. Which property fails?
2. Does this necessarily cause deadlock?
3. Give a FIFO-ticket rule and bound from Pi’ s successful atomic ticket acquisition, assuming n fixed participants, at most one outstanding request per participant, no wraparound, terminating holders, and eventual scheduling.

OS05-E02 Bounded buffer

Capacity3, empty=0, full=3, mutex available.

1. Which semaphore must the producer wait first?
2. Give the consumer's signal order after removal.
3. Why must the producer not hold mutex while waiting empty?

OS05-E03 Condition variable

Waiter executes if(!ready) wait(cv); use(buffer). Notifier sets ready=true then signals.

1. Why can if be insufficient?
2. Give the correct waiting form.
3. When notification precedes inspection, consider both ready remaining true and another thread changing it back to false.

OS05-E04 Peterson correctness and ordering

Two threads use sequentially consistent atomic flag[2], initially false, and turn∈{0,1}. For liveness, assume relevant contenders are scheduled and each critical section terminates. Entry writes flag[i]=true, turn=j, then waits while(flag[j] && turn==j). Exit clears flag[i].

1. Prove exclusion by comparing the final turn writes.
2. From Pi completing flag[i]=true and turn=j, prove Pj can enter at most once before Pi. Why does counting from the earlier flag write alone differ?
3. Explain all three properties under sequential consistency.
4. Which proof step fails with non-atomic variables and compiler reordering?

OS05-E05 Semaphore states

Capacity 3, empty=3, full=0, mutex=1. Produce A, B, C; consume A; produce D. For part 2 use a specified signed-counter implementation with atomic direct handoff reserved for the chosen waiter.

1. Trace empty, full, mutex, and contents after each operation.
2. Why does a binary semaphore not admit a second wait?
3. If a producer blocks on empty while holding mutex, can the consumer remove an item? Give the wait edges.
4. Define the in-flight producer and consumer reservations and derive the capacity invariant away from quiescent boundaries.

OS05-E06 Mesa and lost wakeup

A monitor protects ready=false and buffer. Waiter uses while(!ready) wait(cv). Notifier holds the same monitor, sets ready=true, and signals.

1. Why must releasing the monitor and joining the queue be atomic?
2. If notification occurs before inspection, distinguish ready still being true from another thread consuming it and setting it false.
3. Under Mesa, another thread resets ready=false before the awakened waiter reacquires. May the waiter use buffer?
4. Compare Hoare/Mesa handoff and correctness when the signaler establishes the predicate before signaling; for the Hoare case assume no spurious wakeup or timeout.

OS05-E07 The notified consumer does not receive the only item

Use Mesa semantics, mutex m, condition variable cv, and initially empty q. C1/C2 each run `lock(m); while(q.empty()) wait(cv,m); pop(q); unlock(m)`. P runs `lock(m); push(q,A); signal(cv); unlock(m)` once. No spurious wakeup occurs; m is not FIFO.

Events are T0: C1 waits; T1: P holds m and inserts A; T2: P still owns m and notifies C1; T3: P unlocks, then C2 completes get and unlocks; T4: C1 reacquires m; T5: C1's next action.

1. List m owner, q, CV waiters, and notified-but-not-reacquired set E after each of six events T0–T5. The final event is C1's next action.
2. If while becomes if, identify the first invalid operation. Why is this neither a spurious wakeup nor a lost wakeup?
3. Split unlock from later wait registration. Give three steps that leave C1 asleep despite nonempty q. Is that legal pthread_cond_wait usage?
4. If both consumers are already waiting and P broadcasts after inserting one A, may both remove an item? Does while guarantee fairness or eventual completion?

OS05-E08 Bakery selection and equal tickets

Three participants run the chapter's Bakery protocol using sequentially consistent atomic shared variables and nonoverflowing ticket numbers; initially none has a ticket.

1. P0 and P1 both see maximum 0 before either publishes, and both select 1. Which pair has priority, and why can tickets be equal?
2. If waiting for `choosing[j]` is omitted, give an interleaving in which both enter.
3. With at most one outstanding request per participant, how many other existing or choosing requests can precede Pi after it finishes choosing? What liveness assumptions are needed for eventual entry?

References and attribution

Lin Cing, handwritten operating systems notes. Silberschatz, Galvin and Gagne, Operating System Concepts, 9th ed., Chapter 5. Lamport, A New Solution of Dijkstra's Concurrent Programming Problem, 1974. pthread_cond_wait(3p); Windows spin locks; Windows critical sections; Linux lock types; OpenMP critical.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study